

MoF 连续权重网络：从离散查表到连续函数

将 $\lambda_{k,t,s}$ 推广为 $\lambda_k = f_\psi(\mathbf{c}_{\text{prompt}}, \mathbf{c}_{\text{time}})$

Flow Factory

1 动机与问题定义

1.1 当前架构的局限

当前 MoF 将混合权重存储为离散查表张量：

$$\mathbf{\Lambda} \in \mathbb{R}^{K \times T \times S}, \quad w_{k,t,s} = \frac{\exp(\Lambda_{k,t,s}/\tau)}{\sum_{k'=1}^K \exp(\Lambda_{k',t,s}/\tau)} \quad (1)$$

其中 K = teacher 数量, T = 训练时的 denoising 步数, S = prompt 来源集合数量。
问题：

1. **Timestep** 绑定：权重按 step index 查表，改变推理步数时无法正确映射；
2. **Prompt** 离散化：按 source 分类 (geneval/pickscore/ocr)，无法处理未见过的 prompt 类型或同一 source 内不同 prompt 的最优混合差异；
3. 泛化能力： $K \times T \times S$ 参数是纯记忆 (lookup table)，不具备插值或外推能力。

1.2 目标

将混合权重从离散 lookup table 推广为连续函数：

$$w_k(t, \mathbf{c}) = \frac{\exp(f_\psi^{(k)}(t, \mathbf{c})/\tau)}{\sum_{k'=1}^K \exp(f_\psi^{(k')}(t, \mathbf{c})/\tau)} \quad (2)$$

其中：

- $t \in [0, 1]$: 连续时间 (归一化 timestep)，支持任意推理步数；
- $\mathbf{c}$: prompt 条件向量 (可以是 pooled text embedding, 也可以是中间隐表示)；
- $f_\psi: \mathbb{R} \times \mathbb{R}^{d_c} \rightarrow \mathbb{R}^K$: 权重预测网络，参数为 ψ 。

2 数学框架

2.1 符号定义

符号	含义
K	Teacher 模型数量
$v_k(\mathbf{x}_t, t, \mathbf{c})$	第 k 个 teacher 在 $(\mathbf{x}_t, t, \mathbf{c})$ 处的预测速度 (frozen)
$w_k(t, \mathbf{c}; \psi)$	第 k 个 teacher 的混合权重 (learnable)
$\bar{v}(\mathbf{x}_t, t, \mathbf{c})$	组合速度 = $\sum_{k=1}^K w_k(t, \mathbf{c}) \cdot v_k(\mathbf{x}_t, t, \mathbf{c})$
τ	Softmax 温度
ψ	权重网络参数
$\mathbf{c}_{\text{pool}}$	Pooled text embedding $\in \mathbb{R}^d$ (来自 text encoder)
$\mathbf{e}_t$	Timestep embedding $\in \mathbb{R}^{d_t}$

2.2 组合速度

在每个 denoising step，组合速度定义为：

$$\bar{v}(\mathbf{x}_t, t, \mathbf{c}; \psi) = \sum_{k=1}^K w_k(t, \mathbf{c}; \psi) \cdot \text{sg}(v_k(\mathbf{x}_t, t, \mathbf{c})) \quad (3)$$

$\text{sg}(\cdot)$ 表示 stop-gradient (teacher 速度冻结，梯度仅流过权重)。

2.3 GRPO 损失 (不变)

GRPO 优化目标不变：

$$\text{ratio}(t) = \exp(\log p_\psi(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{c}) - \log p_{\psi_{\text{old}}}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{c})) \quad (4)$$

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}_{t, \mathbf{c}} [\mathcal{A} \cdot \min(\text{ratio}(t), \text{clip}(\text{ratio}(t), 1 \pm \epsilon))] \quad (5)$$

梯度链：

$$\psi \xrightarrow{f_\psi} \text{logits} \xrightarrow{\text{softmax}} w_k \xrightarrow{\text{combine}} \bar{v} \xrightarrow{\text{sched.step}} \log p_\psi \xrightarrow{\text{exp}} \text{ratio} \xrightarrow{-\mathcal{A} \cdot} \mathcal{L}$$

2.4 梯度推导

设 $\ell_k = f_\psi^{(k)}(t, \mathbf{c})$ 为第 k 个 teacher 的 logit 输出，则：

$$\frac{\partial w_k}{\partial \ell_j} = w_k(\delta_{kj} - w_j)/\tau \quad (6)$$

对 $\bar{v}$ 的梯度：

$$\frac{\partial \bar{v}}{\partial \ell_j} = \frac{1}{\tau} \sum_{k=1}^K w_k(\delta_{kj} - w_j) \cdot v_k = \frac{w_j}{\tau} (v_j - \bar{v}) \quad (7)$$

这说明 logit ℓ_j 的梯度方向与 teacher j 的速度偏离组合速度的方向有关——当 advantage 为正时，梯度会倾向于增大那些能使 log-prob 增大的 teacher 的权重。

对网络参数 ψ 的梯度（通过链式法则）：

$$\frac{\partial \mathcal{L}}{\partial \psi} = \sum_{k=1}^K \frac{\partial \mathcal{L}}{\partial \ell_k} \cdot \frac{\partial \ell_k}{\partial \psi} = \sum_{k=1}^K \frac{\partial \mathcal{L}}{\partial \ell_k} \cdot \nabla_\psi f_\psi^{(k)}(t, \mathbf{c}) \quad (8)$$

其中 $\frac{\partial \mathcal{L}}{\partial \ell_k}$ 由 GRPO ratio-loss 反传得到（与当前离散版完全一致）。

3 现代 DiT 模型的 Conditioning 机制分析

在设计 MoF 权重网络之前，需要理解主流 DiT 模型如何注入条件信息。不同模型采用不同策略，我们需要一个通用方案。

3.1 Flux.1 (Black Forest Labs)

Text 注入：

- 使用 T5-XXL (dense text features) + CLIP (cross-modal alignment) 双编码器
- 无 **pooled output**：Flux.1 的 MMDiT 直接通过 joint attention 处理 text token 序列
- Text tokens 与 image tokens 拼接后通过 joint self-attention 交互

Timestep 注入:

- 通过 **adaLN-Zero** (Adaptive Layer Normalization) 将 timestep embedding 注入每个 block
- timestep $\rightarrow$ SinusoidalEmb $\rightarrow$ MLP $\rightarrow (\gamma, \beta, \alpha)$ 用于调制 LayerNorm
- 无显式的 **pooled_projections** 输入

对 **MoF** 的启示: Flux.1 没有 pooled text embedding, 只有 text token 序列。若要支持 Flux, 必须从 **prompt_embeds** 序列中提取条件信息。

3.2 Qwen-Image / Wan (Alibaba)

Text 注入:

- 使用 Qwen2.5-VL (7B) 作为 text encoder, 输出 dense token 序列
- 通过 cross-attention 在每个 DiT block 中注入
- 无 **pooled output**: 直接用完整 hidden states 做 cross-attention
- MS-RoPE 实现 text token 与 image patch 的空间对齐

Timestep 注入:

- 标准 adaLN 机制: timestep embedding 通过 MLP 生成 scale/shift 参数

对 **MoF** 的启示: Wan 同样没有 pooled embedding, 只有序列。路由网络必须能处理变长 token 序列。

3.3 SD3.5 (Stability AI)

Text 注入:

- CLIP-L + CLIP-G + T5-XXL 三编码器
- 有 **pooled output**: $\mathbf{c}_{\text{pool}} \in \mathbb{R}^{2048}$ (CLIP pooled 拼接)
- Token 序列通过 MMDiT joint attention 注入
- Pooled embedding 通过 **pooled_projections** 输入, 与 timestep embedding 拼接后送入 adaLN

Timestep 注入:

- Sinusoidal $\rightarrow$ MLP $\rightarrow$ 与 **pooled_projections** 拼接 $\rightarrow$ adaLN modulation

3.4 通用性分析

模型	pooled_embeds	prompt_embeds 序列	DiT hidden states
SD3.5	✓ ($\mathbb{R}^{2048}$)	✓ ($\mathbb{R}^{L \times 4096}$)	✓
Flux.1	×	✓ ($\mathbb{R}^{L \times 4096}$)	✓
Wan/Qwen	×	✓ ($\mathbb{R}^{L \times d}$)	✓
PixArt- α	×	✓ ($\mathbb{R}^{L \times 4096}$)	✓

结论: **pooled_embeds** 不通用 (仅 SD3 有); **prompt_embeds** 序列所有模型都有; DiT hidden states 所有模型都有。

3.5 DiT Router 相关工作

近期 DiT MoE 路由网络（2024–2025）的关键设计：

1. **MoDE** (Noise-Conditioned Routing)：根据 denoising timestep 的噪声水平选择 expert，实现 expert 在不同噪声阶段的专业化。
2. **EC-DiT** (Expert-Choice)：基于 text complexity 和 image patch 重要性分配 expert，97B 参数 64 experts。
3. **ProMoE** (Prototypical Routing)：用 learnable prototypes 在 latent space 做语义级别路由。
4. **adaLN-single** (PixArt- α)：所有 block 共享一组 timestep-conditioned 调制参数，通过 per-block learnable offset 区分。

这些工作的共同模式是：**router** 输入为当前 **step** 的 **hidden states** (或条件 **embedding**)，输出为 **per-expert** 权重。

4 权重网络架构设计

基于上述分析，权重网络的输入需要满足跨模型通用性。我们分析可用信号源及对应方案。

4.1 可用信号源

1. **prompt_embeds** (所有模型都有)：text encoder 输出的 token 序列 $\mathbf{H} \in \mathbb{R}^{B \times L \times d_{\text{text}}}$
2. **timestep** (所有模型都有)：连续时间 $t \in [0, 1]$ 或 scheduler 整数值
3. **pooled_prompt_embeds** (仅 SD3)：全局 text summary $\in \mathbb{R}^{B \times d_{\text{pool}}}$
4. **DiT hidden states** (所有模型都有)：中间层 $\mathbf{Z}_l \in \mathbb{R}^{B \times (H'W' + L) \times d_{\text{model}}}$

4.2 输入表示方案

4.2.1 方案 A: Attention Pooling on prompt_embeds (推荐)

对变长 text token 序列做 attention-based pooling，产出固定维度的 prompt summary：

$$\mathbf{c} = \text{AttnPool}(\mathbf{H}) = \frac{\sum_{i=1}^L \alpha_i \mathbf{h}_i}{\sum_{i=1}^L \alpha_i}, \quad \alpha_i = \exp(\mathbf{q}^\top \mathbf{h}_i / \sqrt{d}) \quad (9)$$

其中 $\mathbf{q} \in \mathbb{R}^{d_{\text{text}}}$ 是 learnable query vector。

优势：

- 通用：所有模型都有 prompt_embeds (无论是否有 pooled output)
- 轻量：仅需 1 个 learnable query vector + projection ($\sim d$ 参数)
- 有效：attention pooling 已被验证优于 mean pooling (Set Transformer, Perceiver 等)
- 可降级：对有 pooled_embeds 的模型，可直接用 pooled_embeds bypass attention pooling

4.2.2 方案 B: Mean Pooling on prompt_embeds

$$\mathbf{c} = \frac{1}{L} \sum_{i=1}^L \mathbf{h}_i \quad (10)$$

优势：无额外参数。劣势：信息损失大，长序列被稀释。

4.2.3 方案 C: DiT Hidden States (最强但最贵)

从 DiT 的某个中间层提取 hidden states 作为条件:

$$\mathbf{c} = \text{Pool}(\mathbf{Z}_{l^*}[\text{text_tokens}]) \quad (11)$$

即取 DiT 第 l^* 层的 text-position hidden states 做 pooling。

优势: 包含图像-文本交互后的信息 (text tokens 已经 attend to image patches)。

劣势:

- 需要在 **teacher forward** 之前获取 hidden states $\rightarrow$ 需要额外一次 partial forward
- 或者使用前一步的 **hidden states** (one-step delay), 牺牲少量信息
- 增加计算开销

适用场景: 当 prompt_embeds 不足以区分样本时 (如所有样本用相同 prompt template)。

4.2.4 Timestep 条件 (统一方案)

所有方案共享同一种 timestep encoding:

$$\mathbf{e}_t = \text{MLP}(\text{SinusoidalPosEmb}(t)) \in \mathbb{R}^{d_h} \quad (12)$$

其中 t 为 scheduler 的归一化时间值 $\in [0, 1]$ 。MLP 将正弦编码映射到网络隐层维度。

4.3 网络结构方案

4.3.1 方案 I: adaLN-style MLP Router (推荐)

受 PixArt- α 的 adaLN-single 和 MoDE noise-conditioned routing 启发, 采用类似 adaLN 的条件注入方式:

$$\mathbf{e}_t = \text{SiLU}(\text{Linear}(\text{SinusoidalEmb}(t))) \in \mathbb{R}^{d_h} \quad (13)$$

$$\mathbf{c} = \text{AttnPool}(\text{prompt_embeds}) \in \mathbb{R}^{d_{\text{text}}} \quad (14)$$

$$\mathbf{c}_{\text{proj}} = \text{Linear}(\mathbf{c}) \in \mathbb{R}^{d_h} \quad (15)$$

$$(\text{adaLN modulation}) \quad \gamma, \beta = \text{chunk}(\text{Linear}(\mathbf{e}_t), 2) \quad (16)$$

$$\mathbf{h} = \gamma \odot \mathbf{c}_{\text{proj}} + \beta \quad (\text{time modulates text}) \quad (17)$$

$$\mathbf{h} = \text{SiLU}(\text{Linear}(\mathbf{h})) \quad (18)$$

$$\ell = \text{Linear}(\mathbf{h}) \in \mathbb{R}^K \quad (19)$$

核心思想: timestep 通过 adaLN 调制 text embedding, 使得同一 prompt 在不同去噪阶段产生不同权重。这直接对应 MoDE 的发现——不同噪声阶段需要不同 expert 专业化。

参数量: 设 $d_h = 256$, $d_{\text{text}} = 4096$, $K = 3$:

$$\underbrace{256}_{\text{SinEmb}} + \underbrace{256 \cdot 256}_{e_t \text{ MLP}} + \underbrace{d_{\text{text}}}_{q_{\text{attn}}} + \underbrace{4096 \cdot 256}_{c_{\text{proj}}} + \underbrace{256 \cdot 512}_{\gamma, \beta} + \underbrace{256 \cdot 256}_{\text{MLP}} + \underbrace{256 \cdot 3}_{\text{out}} \approx 1.2\text{M}$$

4.3.2 方案 II: 简化 MLP (基线)

直接拼接所有条件:

$$f_\psi(t, \mathbf{c}) = \text{MLP}([\mathbf{e}_t; \text{Linear}(\text{AttnPool}(\mathbf{H}))]) \in \mathbb{R}^K \quad (20)$$

$$\mathbf{h}_0 = [\mathbf{e}_t; \mathbf{c}_{\text{proj}}] \in \mathbb{R}^{2d_h} \quad (21)$$

$$\mathbf{h}_1 = \text{SiLU}(\text{Linear}_{2d_h \rightarrow d_h}(\mathbf{h}_0)) \quad (22)$$

$$\ell = \text{Linear}_{d_h \rightarrow K}(\mathbf{h}_1) \quad (23)$$

优势: 最简单, debug 容易。劣势: time 和 text 的交互只能依赖 MLP 隐式学习。

4.3.3 方案 III: Cross-Attention Router

以 teacher learnable embedding 为 KV, 条件向量为 Q:

$$\mathbf{q} = W_q[\mathbf{e}_t; \mathbf{c}] \in \mathbb{R}^{d_k} \quad (24)$$

$$\mathbf{K} = W_k \cdot \mathbf{E}_{\text{teachers}} \in \mathbb{R}^{K \times d_k} \quad (\text{learnable teacher embeddings}) \quad (25)$$

$$\ell = \frac{\mathbf{K} \cdot \mathbf{q}}{\sqrt{d_k}} \in \mathbb{R}^K \quad (26)$$

优势: 结构化, 可解释 (teacher embedding 空间可视化), 参数高效。

劣势: 当 K 小 ($= 3$) 时, teacher embedding 可能退化为 bias 向量。

4.3.4 方案 IV: DiT Hidden State Router (最强泛化)

利用 DiT 中间层的 hidden states (已包含 image-text 交互信息):

$$\mathbf{Z}_l = \text{DiT.layer}_l(\mathbf{x}_t, \mathbf{c}) \quad (\text{hook from teacher forward}) \quad (27)$$

$$\mathbf{h}_{\text{text}} = \text{AttnPool}(\mathbf{Z}_l[\text{text positions}]) \quad (28)$$

$$\ell = \text{MLP}([\mathbf{h}_{\text{text}}; \mathbf{e}_t]) \quad (29)$$

适用场景: prompt_embeds 不能充分区分样本时。但增加实现复杂度 (需 forward hook 或 dual pass)。

4.4 推荐方案与理由

最终推荐：方案 I (adaLN-style MLP + AttnPool)

架构：

1. **Text** 条件：AttnPool(prompt_embeds) $\rightarrow$ Linear projection
2. **Time** 条件：SinusoidalEmb(t) $\rightarrow$ MLP $\rightarrow$ 产生 γ, β
3. 融合：Time adaLN-modulates Text $\rightarrow$ MLP $\rightarrow$ K logits

选择理由：

1. 通用性：仅依赖 prompt_embeds + timestep，所有 DiT 模型都有这两个信号
2. 符合 **DiT** 范式：adaLN 是所有主流 DiT (SD3, Flux, PixArt, Wan) 注入 timestep 的标准方式。我们的 router 复用这一设计，使其行为与 DiT 内部一致
3. 噪声阶段专业化：Time modulates text embedding $\rightarrow$ 同一 prompt 在去噪早期（全局结构）vs 晚期（细节纹理）选择不同 teacher 权重。这对应 MoDE 的核心发现
4. 参数效率： $\sim 1.2\text{M}$ 参数（vs teacher 几十亿参数），训练快速收敛
5. 可降级：当模型提供 pooled_embeds 时，可跳过 AttnPool 直接使用 pooled_embeds

方案	通用性	参数量	表达力	训练稳定性	推理开销
II. 简化 MLP	高	$\sim 1\text{M}$	中	高	$< 1\text{ms}$
I. adaLN Router	高	$\sim 1.2\text{M}$	强	高	$< 1\text{ms}$
III. Cross-Attn Router	高	$\sim 0.5\text{M}$	中	中	$< 1\text{ms}$
IV. Hidden State Router	最高	$\sim 1.5\text{M}$	最强	中	$\sim 5\text{ms}$

5 训练流程

5.1 与当前 GRPO 的对比

	当前（离散 LUT ）	扩展（连续网络）
可学习参数	$\mathbf{\Lambda} \in \mathbb{R}^{K \times T \times S}$	网络参数 ψ
权重获取	$w = \text{softmax}(\mathbf{\Lambda}[:, t_{\text{idx}}, s] / \tau)$	$w = \text{softmax}(f_{\psi}(t, \mathbf{c}) / \tau)$
优化器输入	$\mathbf{\Lambda}$ (单 Parameter)	ψ (网络所有参数)
梯度流	$\mathcal{L} \rightarrow w \rightarrow \mathbf{\Lambda}$	$\mathcal{L} \rightarrow w \rightarrow \ell \rightarrow \psi$
推理时输入	step index t_{idx} , source s	连续 t , prompt embedding $\mathbf{c}$

5.2 训练伪代码

Algorithm 1 MoF-GRPO with Continuous Weight Network

Require: 冻结的 teacher 模型 $\{v_k\}_{k=1}^K$, 权重网络 f_ψ , text encoder TE

Require: Scheduler, 学习率 η , clip range ϵ , advantage $\mathcal{A}$

```

1: for epoch = 1, 2, ... do
2:   // Phase 1: Sampling (on-policy)
3:   for each batch of prompts  $\{\mathbf{p}_b\}_{b=1}^B$  do
4:      $\mathbf{c}_b \leftarrow \text{TE}(\mathbf{p}_b).\text{pooled}$  ▷ Prompt conditioning
5:      $\mathbf{x}_T \sim \mathcal{N}(0, I)$  ▷ Initial noise
6:     for  $i = 0, 1, \dots, N - 1$  do ▷ Denoising steps
7:        $t \leftarrow \text{Scheduler.timesteps}[i]$  ▷ Continuous time value
8:        $\ell \leftarrow f_\psi(t, \mathbf{c}_b)$  ▷ Predict logits, shape (K,)
9:        $\mathbf{w} \leftarrow \text{softmax}(\ell/\tau)$  ▷ Normalize weights
10:      for  $k = 1, \dots, K$  do
11:         $\hat{v}_k \leftarrow v_k(\mathbf{x}_t, t, \mathbf{c}_b)$  ▷ Frozen teacher forward
12:      end for
13:       $\bar{v} \leftarrow \sum_k w_k \cdot \hat{v}_k$  ▷ Combine velocities
14:       $\mathbf{x}_{t-1}, \log p_{\text{old}} \leftarrow \text{Scheduler.step}(\bar{v}, t, \mathbf{x}_t)$ 
15:      Store:  $(\mathbf{x}_t, t, \mathbf{c}_b, \log p_{\text{old}})$ 
16:    end for
17:  end for
18:  Compute rewards and advantages  $\mathcal{A}_b$  for each trajectory
19:
20:  // Phase 2: Optimization (GRPO)
21:  for each stored tuple  $(\mathbf{x}_t, t, \mathbf{c}_b, \log p_{\text{old}}, \mathcal{A}_b)$  do
22:     $\ell \leftarrow f_\psi(t, \mathbf{c}_b)$  ▷ Re-compute with current  $\psi$ 
23:     $\mathbf{w} \leftarrow \text{softmax}(\ell/\tau)$ 
24:    for  $k = 1, \dots, K$  do
25:       $\hat{v}_k \leftarrow \text{sg}(v_k(\mathbf{x}_t, t, \mathbf{c}_b))$  ▷ Detached
26:    end for
27:     $\bar{v} \leftarrow \sum_k w_k \cdot \hat{v}_k$  ▷ Grad flows through  $w_k$ 
28:     $\log p_{\text{new}} \leftarrow \text{Scheduler.step}(\bar{v}, t, \mathbf{x}_t, \mathbf{x}_{t-1}).\text{log\_prob}$ 
29:     $r \leftarrow \exp(\log p_{\text{new}} - \log p_{\text{old}})$  ▷ Policy ratio
30:     $\mathcal{L} \leftarrow -\mathcal{A}_b \cdot \min(r, \text{clip}(r, 1 \pm \epsilon))$ 
31:     $\psi \leftarrow \psi - \eta \cdot \nabla_\psi \mathcal{L}$  ▷ Update network
32:  end for
33: end for

```

5.3 关键设计选择

1. **On-policy 一致性**: Sampling 和 Optimization 必须使用同一个 f_ψ (当前版本) 来保证 $\text{ratio} = 1$ at iteration start。这与我们之前修复的 `off_policy=False` 约束完全一致。
2. **Prompt conditioning** 的来源: sampling 阶段 text encoder 已经运行过, pooled embedding 可以直接复用 (零额外成本)。
3. **Timestep** 的传入: 使用 scheduler 的连续时间值 (而非 step index), 这样权重网络天然支持任意步数推理。
4. 训练信号: 梯度来源与离散版完全相同 (GRPO ratio loss), 唯一变化是 $\partial \ell / \partial \psi$ 由网络反传提供 (替代了对 lookup table 的直接更新)。

6 实现考量

6.1 与现有代码的对接

当前 MoFMixingModule 需替换为:

```
class MoFWeightNetwork(nn.Module):
    """adaLN-style continuous weight prediction network.

    Inputs: prompt_embeds (sequence) + timestep (scalar)
    Output: (K,) or (K, B) mixing weights

    Architecture:
    1. AttnPool(prompt_embeds) -> c_proj [text summary]
    2. SinusoidalEmb(t) -> MLP -> (gamma, beta) [time modulation]
    3. gamma * c_proj + beta [adaLN fusion]
    4. MLP -> K logits -> softmax
    """
    def __init__(self, K, d_text, d_hidden=256, d_time=256, tau=1.0):
        super().__init__()
        self.K = K
        self.tau = tau

        # Attention pooling for prompt_embeds (universal)
        self.attn_query = nn.Parameter(torch.randn(1, 1, d_text))
        self.c_proj = nn.Linear(d_text, d_hidden)

        # Timestep embedding -> adaLN parameters
        self.time_embed = nn.Sequential(
            SinusoidalPosEmb(d_time),
            nn.Linear(d_time, d_hidden),
            nn.SiLU(),
        )
        self.adaLN_modulation = nn.Linear(d_hidden, 2 * d_hidden) # gamma, beta

        # Output MLP
        self.mlp = nn.Sequential(
            nn.SiLU(),
            nn.Linear(d_hidden, d_hidden),
            nn.SiLU(),
            nn.Linear(d_hidden, K),
        )
        # Zero-init output for stable start (uniform weights)
        nn.init.zeros_(self.mlp[-1].weight)
        nn.init.zeros_(self.mlp[-1].bias)

    def forward(self, t, prompt_embeds, pooled_prompt_embeds=None):
        """
        Args:
            t: (B,) timestep (normalized or raw scheduler value)
            prompt_embeds: (B, L, d_text) text token sequence
            pooled_prompt_embeds: (B, d_pool) optional, bypass AttnPool
        Returns:
```

```

        weights: (K, B) mixing weights
"""
# Text conditioning
if pooled_prompt_embeds is not None:
    c = self.c_proj(pooled_prompt_embeds) # (B, d_hidden)
else:
    # Attention pooling over token sequence
    attn_scores = (self.attn_query @ prompt_embeds.transpose(-1, -2))
    attn_weights = F.softmax(attn_scores / (prompt_embeds.shape[-1] ** 0.5), dim=-1)
    c_pooled = (attn_weights @ prompt_embeds).squeeze(1) # (B, d_text)
    c = self.c_proj(c_pooled) # (B, d_hidden)

# Time conditioning -> adaLN modulation
e_t = self.time_embed(t) # (B, d_hidden)
gamma, beta = self.adaLN_modulation(e_t).chunk(2, dim=-1)

# adaLN fusion: time modulates text
h = gamma * c + beta # (B, d_hidden)

# Predict logits
logits = self.mlp(h) # (B, K)
weights = F.softmax(logits / self.tau, dim=-1) # (B, K)
return weights.T # (K, B) for velocity stacking convention

```

6.2 Sampling 阶段的变化

`_mof_inference_context` 中, 将:

```
w_i = set_weights[:, t_idx] # (K,) from lookup
```

替换为:

```
w_i = self._weight_network(t_normalized, prompt_embeds, pooled) # (K, B)
```

6.3 Optimization 阶段的变化

`grpo.py` 的 `_combine_velocities_per_sample` 调用处, 将:

```

current_weights = self._get_lambda_weights(self._lambda_logits) # (K, T, S)
v_combined = self._combine_velocities_per_sample(
    teacher_velocities, timestep_index, current_weights, set_ids
)

```

替换为:

```

# t: (B,) actual timestep values, c_pool: (B, d_pool)
weights = self._weight_network(t, batch['pooled_prompt_embeds']) # (K, B)
# Expand for spatial dims and combine
w_expanded = weights.unsqueeze(-1).unsqueeze(-1).unsqueeze(-1) # (K, B, 1, 1, 1)
v_combined = (w_expanded * teacher_velocities).sum(dim=0) # (B, C, H, W)

```

6.4 训练超参建议

- 学习率: 10^{-4} 至 5×10^{-4} (比离散版的 0.05 小很多, 因为网络参数相互耦合)

- 权重衰减: 10^{-4} (防止权重爆炸)
- 温度 τ : 初始 1.0, 可 anneal 到 0.5
- 初始化: 输出层 bias 初始化为 0 (uniform weights at start)
- **Gradient clipping**: $\text{max_grad_norm} = 1.0$

7 渐进式实现路线

阶段 1 (验证可行性):

1. 保持离散 LUT 不变, 仅在推理脚本中用 MLP 拟合已训练好的离散权重, 验证连续化不损失性能。

阶段 2 (端到端训练):

1. 替换 `MoFMixingModule` 为 `MoFWeightNetwork`
2. 修改 `optimizer` 为网络参数
3. 在现有 GRPO 框架下训练, 验证收敛性

阶段 3 (泛化验证):

1. 用训练时未见过的 prompt 测试 (OOD generalization)
2. 用不同推理步数测试 (timestep generalization)
3. 与离散版做 A/B 对比